Agentic UI — Feature Definition

IOT 236332, Summer 2026 · Project: *Agentic UI* (Project list A) · 2026-09-08

Team: _____ Repository: _____

1. What the project does

A person describes the touch interface they want in plain language — for example *"an interface that adds two numbers, with a Calculate button and a result display"*. The system uses a language model to write firmware for that screen, compiles it, flashes it to an ESP32 driving a 320×240 touch panel, then **photographs the real panel with a webcam** and judges whether the screen that actually appeared matches what was asked for. If it does not, it corrects itself and tries again.

The point of the camera is that nothing is taken on trust: the evidence that the UI works is a photograph of the physical glass, not a simulator render.

2. Parts of the system

#	Part	Runs on	Responsibility
1	Firmware harness *(fixed)*	ESP32	Display init, touch reading and calibration, serial control protocol, safe-mode fallback
2	UI layer *(regenerated)*	ESP32	The widget table and drawing code — the only part the agent rewrites
3	Orchestrator	PC (Python)	Runs the request → generate → build → flash → photograph → judge loop
4	Build service	PC	PlatformIO CLI: compile, capture errors, flash over USB
5	Vision and verification	PC + Gemini	Captures a webcam frame, de-warps the panel, asks the model whether the UI matches the request
6	Artifact store	PC (files + git)	Every iteration's source, photo, build log and verdict, so versions can be compared and restored

Why the firmware has a serial protocol. The board answers `DESC` with a JSON description of every widget it believes it drew, and accepts `TAP <widget>` to inject a press. The loop can therefore compare three independent things: what was *asked for*, what the firmware *thinks* it drew, and what the camera *actually sees* — and a disagreement tells us which layer is broken.

3. User stories

Sorted by significance. Story 1 alone carries roughly half the project's complexity.

#	Story name	As a...	I want...	So that...
1	Describe a UI in words	Designer	to describe the screen I need in everyday language — "two number fields, a Calculate button and a result line" — and have it appear on the physical display	I can get a working touch interface without writing embedded code
2	Hands-off build and flash	Designer	the system to compile the generated code with a command-line toolchain and flash it to the board by itself once the build succeeds	I never open an IDE or press a button to see my request on the panel
3	Proof on real hardware	Designer	to see a photograph of the actual panel running my UI	I can trust I am judging the real device and not a preview

#	Story name	As a…	I want…	So that…
4	Automatic verification	Designer	the system to decide by itself whether the screen matches my request — texts, colours, element positions, and anything missing or cut off at the edge	I do not have to inspect every attempt manually
5	Repair of build failures	Designer	the system to feed the compiler's error messages back to the model, get a fix and rebuild	a syntax error in generated code never reaches me as a failure
6	Self-correction on visual mismatch	Designer	the system to update the code, rebuild, reflash and re-photograph when the picture does not match the request	I get a working screen instead of an error
7	Bounded retries and honest verdicts	Designer	the loop to stop after a set number of attempts, and to mark a photo it cannot read as *inconclusive* rather than pass or fail	the system never loops forever and never claims a result it did not actually see
8	Conversational refinement	Designer	to say "make the buttons bigger" and get an updated screen	refining the interface is a conversation rather than a rewrite
9	Touch is proven, not assumed	Designer	each control to be pressed and its effect confirmed	I know the interface is usable and not just a correct-looking picture
10	Multi-screen UIs and value entry	Designer	to ask for several screens with navigation between them, and for numeric entry (stepper, slider or keypad)	I can describe a real application and not only a single static screen
11	Understandable failures	Designer	a plain-language explanation when a screen cannot be built	I can tell whether to rephrase my request or ask for help
12	Version history	Product owner	to browse earlier versions with their photos and return to one	a bad iteration never loses work that was already good
13	Run record	Product owner	every run to keep the original request, each code version, the build logs, the photos, the number of repair attempts and the final status	I can audit exactly how a delivered screen came to be
14	Evidence report	Product owner	a report listing which checks passed and failed, with the photos	I have evidence the delivered UI meets the request
15	Camera setup mode	Technician	guided help finding a camera position and resolution with no moiré	the photographs the system judges are trustworthy
16	Touch calibration	Technician	to calibrate the panel once and have it remembered across power cycles	taps land where fingers actually press
17	Recovery mode	Technician	the board to fall back to a known-good screen when a generated UI crashes or hangs	the device never needs a manual rescue flash
18	Offline operation	Designer	the last built UI to keep running when the network or AI service is unavailable	a demo does not depend on connectivity
19	Configuration	Technician	to set serial port, camera and model choice without editing code	the system can be moved to another PC quickly
20	Diagnostic view	Technician	to see the raw camera frame, raw touch coordinates and widget hit-boxes together	I can tell whether a failure is in the UI, the touch layer or the camera

4. GUI features on the panel

Widgets the generator may use: label · button · toggle · checkbox · slider · numeric stepper · progress bar · text area · tab view · on-screen keypad · list.

Navigation: a UI may consist of more than one screen. The firmware keeps a screen stack, so a generated interface can move between screens and back — for example a settings screen behind a button on the main one. Each screen is described and photographed separately, so verification still applies per screen.

Milestone 1 implements label, button and toggle directly on TFT_eSPI; the richer widgets arrive with LVGL, which runs on top of the same TFT_eSPI driver so nothing is discarded.

Interaction rules the firmware guarantees:

- press and release are separate events; an action fires only if the finger lifts inside the control it went down on, so sliding off cancels
- a pressed control changes appearance while held
- labels and disabled controls never absorb a touch
- the panel repaints only when state changes, so it is static while the camera exposes

Built-in screens: the generated UI · touch-calibration screen · camera test patterns (1-pixel checkerboard, 1-pixel vertical lines, 10-pixel grid, colour bars) · safe-mode screen.

5. Hardware

Item	Qty	Notes
ESP32 DevKit V1	1	course default
ILI9341 2.8" 320×240 SPI display, XPT2046 resistive touch	1	on the course expansion board
Course ESP32 expansion board	1	MISO 19, MOSI 23, SCLK 18, CS 5, DC 15, RST 4, TOUCH_CS 21 — verified against the board document
USB webcam	1	needs ≥1080p and manual focus; resolution chosen empirically against the moiré patterns
Camera stand / fixed mount	1	the panel and camera must not move between iterations
Micro-USB cable, jumpers	—	

No extra sensors are needed: the "sensor" in this project is the camera, and it lives on the PC side because image processing on the microcontroller is explicitly out of scope.

6. Main risks

1. **Moiré and glare** between the LCD pixel grid and the camera sensor grid — mitigated by the built-in test patterns and by fixing resolution and distance before any UI work.
2. **LVGL memory footprint** on the ESP32 — mitigated by keeping the TFT_eSPI path working, so there is always a fallback that fits.
3. **Generated code that does not compile or hangs the board** — mitigated by the safe-mode fallback and by the build log feeding back into the next attempt.
4. **Flash wear from many upload cycles** — mitigated by running the pure UI logic through host unit tests first, so only plausible builds reach the hardware.

7. References

- TFT_eSPI display/touch driver — https://github.com/Bodmer/TFT_eSPI
- LVGL embedded GUI library — https://lvgl.io/ , docs https://docs.lvgl.io/
- PlatformIO Core (headless build/flash) — https://docs.platformio.org/
- Arduino CLI (the alternative we evaluated) — https://arduino.github.io/arduino-cli/

- ESP32 Arduino core — https://github.com/espressif/arduino-esp32
- Gemini API — https://ai.google.dev/
- XPT2046 touch controller datasheet (compatible part) — https://www.ti.com/product/ADS7846
- Unity test framework used by PlatformIO — https://docs.platformio.org/en/latest/advanced/unit-testing/

Course material:

- ICST parts catalog — https://docs.google.com/spreadsheets/d/1V-DETGFDWvvLv6orIYwIOxZJeek2C7KR-pbLn5BZ86k/edit
- ICST GitHub organisation (previous projects) — https://github.com/orgs/ICST-Technion/repositories
- Course knowledge bank, full link index — `IOT_Knowledge_Bank_links.md` in the repository root
- Expansion-board wiring document (course PDF, kept locally); pin map reproduced in `hardware/PINOUT.md`